

Proyecto HPC

Paralelismo y Sistemas Distribuidos
Grado en Ciencia e Ingeniería de Datos
Universidad Politécnica de Cataluña

Trabajo realizado por

Ángel Morales Cirera – DNI: 55182183P

David González Acen – DNI: 41597440P

Abril de 2025

Índice:

1. Introducción
2. Evaluación de la escalabilidad
3. Análisis energético mediante EAR
4. Escalabilidad y métricas de rendimiento
5. Algoritmos y métricas de GPUs
6. Organización del trabajo

1. Introducción:

En este trabajo se estudia la escalabilidad de 3 kernels ejecutados en el Mare Nostrum V utilizando OpenMP y MPI. Se han ejecutado diferentes configuraciones bajo unas ciertas premisas, para así analizar diferentes parámetros de las ejecuciones y poder sacar algunas conclusiones. En cada apartado se entrará en detalles técnicos sobre el análisis de las ejecuciones.

Los objetivos principales del trabajo son estudiar qué kernel es más escalable en OpenMP y en MPI, y estudiar el efecto en el rendimiento y la energía. También se han comprobado parámetros de ancho de banda de memoria y CPI, y se han realizado experimentos también sobre los nodos de GPUs.

En la entrega del trabajo se incluyen todos los scripts utilizados y sus salidas, los kernels, las gráficas (./Graficas) y un archivo ./graficador.py que, al ejecutarlo con Python, lee todos los datos y hace los cálculos que se especifican en cada apartado (excepto los del último apartado). No es necesario ejecutar ese archivo, pero en caso de hacerlo hay que mantener los directorios relativos.

El directorio ./Importado_Mare_Nostrum contiene los scripts y sus entradas (/scripts), salidas (/logs y /csv) y los kernels (/NPB3.4.3-MZ). /csv/csv0 contiene 3 ejecuciones llenando 1 nodo. /csv/csv1 y /csv/vsc2 contiene las salidas de los scripts para 4, 6 y 8 nodos ejecutados dos veces.

El directorio ./GPU contiene el script que se ejecutó en la sesión de laboratorio de GPUs, que genera los datos que se explican en el apartado 5.

2. Evaluación de la escalabilidad

Resumen:

Este es el apartado central del trabajo, con dos objetivos principales. Por un lado, se ha estudiado si la escalabilidad es mejor para OpenMP o para MPI, cuál de estos es más eficiente y cuál es mejor opción bajo unos requisitos de eficiencia. Por otro lado, se ha analizado cada uno de los kernels para ver cuál supone un mayor impacto en el rendimiento en utilizar más o menos threads, cuál es el efecto de OpenMP y MPI en cada uno de ellos y ver cuál es más escalable.

Se han incluido las gráficas del tiempo, el speedup y la eficiencia según el uso de OpenMP vs MPI, previa definición de unos cuantos experimentos que varíen el uso de ambos, distinguiendo los casos según el número total de nodos del supercomputador que se utilicen.

Definición de los experimentos:

Se han utilizado 9 scripts, que se corresponden con las combinaciones de los 3 kernels con 4, 6 y 8 nodos. Es decir, cada script se corresponde con un kernel y un número de nodos fijo. Cada uno de estos scripts incluye casos similares en lo que se refiere a uso de OpenMP y MPI, variando solamente lo necesario para usar más o menos nodos según el caso (variar el kernel solamente supone variar el programa que se usa, sin diferencia en las configuraciones de tareas y cpus).

Los casos intermedios se han definido con la premisa de permitir deducir el efecto de intercambiar OpenMP con MPI. Se necesitan al menos 4 casos intermedios para deducir si el efecto es lineal o no. Teniendo en cuenta que un nodo del Mare Nostrum 5 tiene 112 threads, podemos definir los siguientes casos (usando N como el número de nodos), que mantienen un total de 112 threads en uso en cualquier caso:

- N x 112 tasks y 1 thread por task (caso MPI)
- N x 56 tasks y 2 threads por task
- N x 14 tasks y 8 threads por task
- N x 8 tasks y 14 threads por task
- N x 2 tasks y 56 threads por task
- N tasks y 112 threads por task (caso OpenMP)

Los scripts se han puesto en `Importado_Mare_Nostrum/scripts/`. Están claramente señalados según el número de nodos y el kernel que usan. Además, en esta carpeta están los datos usados como input para las pruebas, y 3 scripts más que sirven para llenar un nodo con el caso MPI. Estos 3 scripts sirven para conseguir el supuesto tiempo en concurrencia que se ha usado para calcular el speedup.

Cada script se ha ejecutado dos veces para poder hacer las medias entre las dos ejecuciones, excepto los casos “concurrentes” de un nodo, que se han ejecutado tres veces.

Tabla de job IDs:

N v kernel >	bt-mz	lu-mz	sp-mz
1	19566132 19572946 19587854	19566140 19572949 19587861	19566145 19572950 19587862
4	19471004 19488923	19471015 19488942	19471035 19488971
6	19471007 19488931	19471021 19488948	19471037 19488975
8	19471014 19488934	19471024 19488953	19471043 19488982

Tabla 1: jobIDs según kernel y número de nodos

Nota: para replicar los experimentos, se debe cargar el módulo ear.

Cada script ejecutado genera dos registros en formato .csv por cada nodo usado. De estos, nos hemos centrado en uno, el que acaba en .time.csv.

Este apartado se centra en analizar el tiempo, speedup y eficiencia. Para esto, se ha calculado el tiempo promedio que tarda en ejecutarse cada configuración de cada script, como la media entre las dos ejecuciones repetidas de el tiempo máximo de cada una, que asimismo se ha calculado como el máximo entre todos los nodos que intervienen.

Por ejemplo, para el caso de 4 nodos con bt-mz, se han calculado por separado los tiempos máximos de entre todos los nodos, configuración a configuración (caso MPI, caso OpenMP, caso de 8 threads por task...) de los dos Jobs: 19471004 y 19488923. Esto da como resultado dos listas de tiempos, el tiempo máximo de cada configuración; y solo falta hacer la media para cada configuración entre las dos repeticiones.

Dada la gran cantidad de cálculos intermedios, lo que se muestra en este estudio son solo las gráficas finales.

Análisis del tiempo:

Tiempos promedios de las ejecuciones con 1 nodo (caso MPI):

bt-mz: 389.8s, lu-mz: 2005.6s, sp-mz: 600.2s

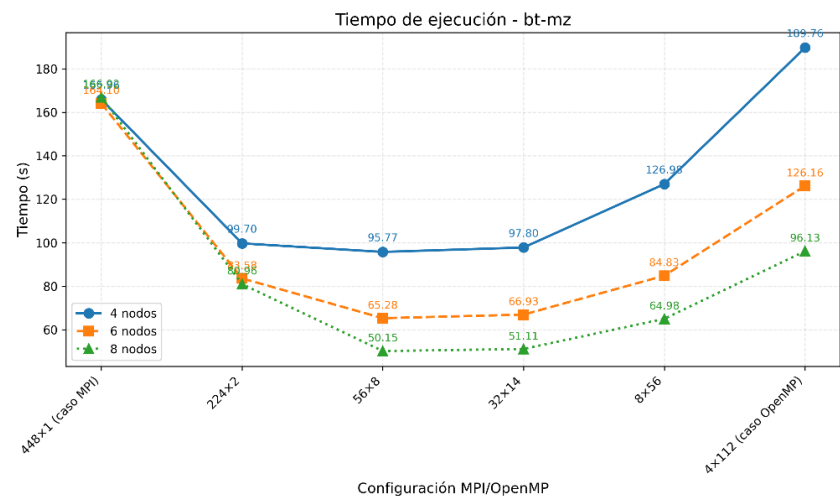


Ilustración 1 - Tiempos de ejecución de bt-mz

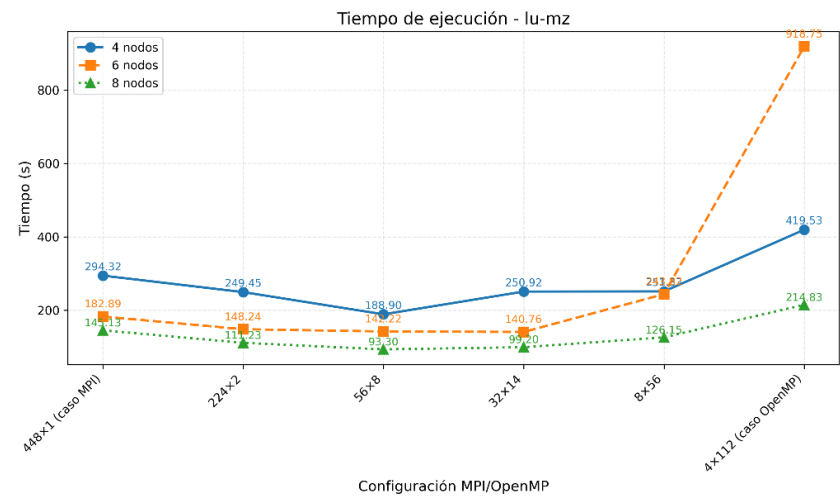


Ilustración 2 - Tiempos de ejecución de lu-mz

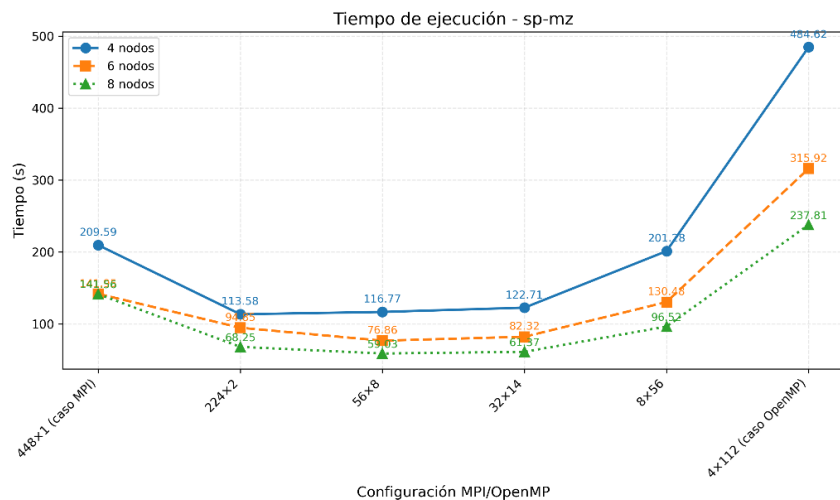


Ilustración 3 - Tiempos de ejecución de sp-mz

Utilizando las ilustraciones 1, 2 y 3, podemos sacar algunas conclusiones respecto a los tiempos de ejecución. Por lo general, incrementar el número de nodos en uso reduce el tiempo de ejecución de cualquier configuración y kernel, excepto lu-mz, que en el caso de OpenMP, se dispara el tiempo con 6 nodos.

Más importante aún es ver como el comportamiento de todos los kernels, tanto en 4, 6 como 8 nodos es similar en cuanto a las configuraciones: cuanto más cerca de los casos MPI u OpenMP, peor. Es decir, por norma general, lo que interesa para conseguir que un programa tarde menos es equilibrar entre MPI y OpenMP, rondando configuraciones de entre dos y catorce threads por task. Por supuesto estos números no tienen por que ser los óptimos en otros programas o computadores, pero podemos asumir que el equilibrio será positivo en la mayoría de casos. Además, si hay que escoger entre el caso MPI o el caso OpenMP, optar por solo MPI suele ser mejor opción.

Análisis del speedup:

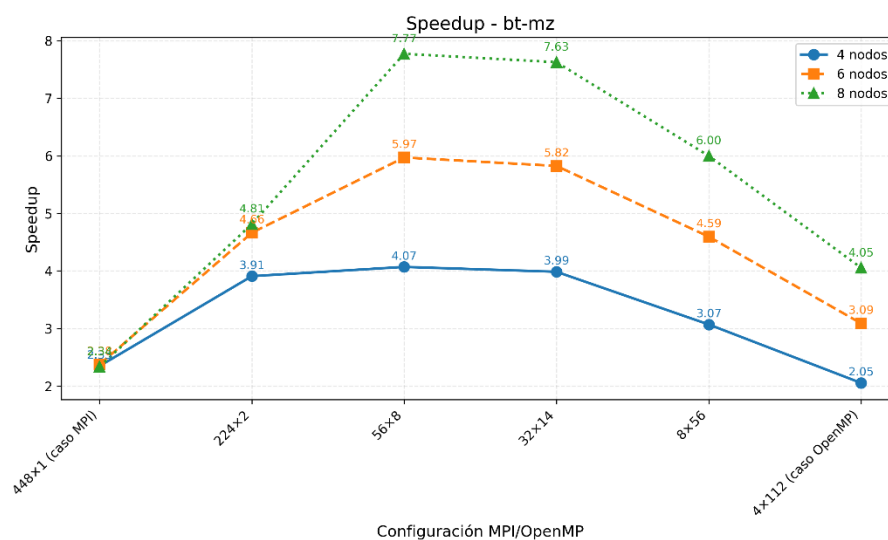


Ilustración 4 – Speedup de bt-mz

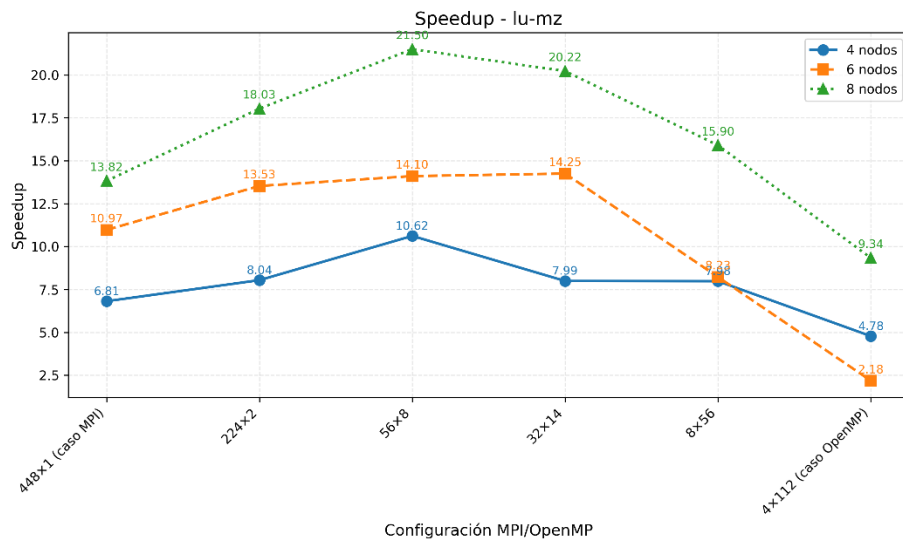


Ilustración 5 - Speedup de lu-mz

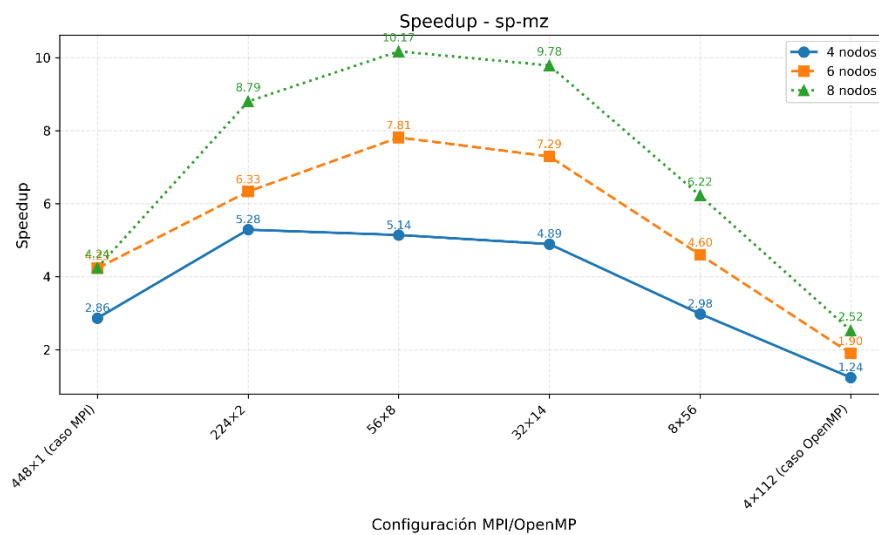


Ilustración 6 - Speedup de sp-mz

El speedup se ha calculado usando como referencia para el tiempo concurrente el tiempo de la ejecución del caso MPI para un nodo. Siendo este el mismo para cada kernel, es de esperar que, si el tiempo se veía reducido al aumentar el número de nodos, el speedup aumenta. Así mismo, el speedup es mayor en los casos mejor balanceados entre MPI y OpenMP.

Análisis de la eficiencia:

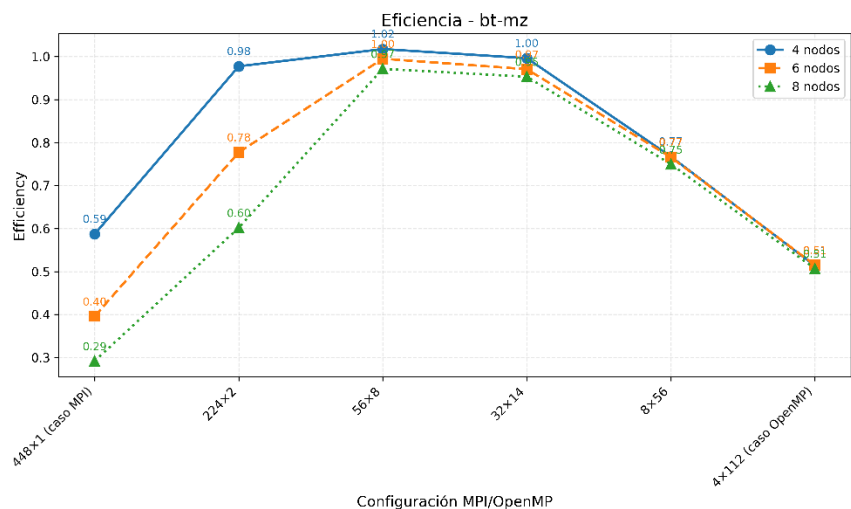


Ilustración 7 - Eficiencia de bt-mz

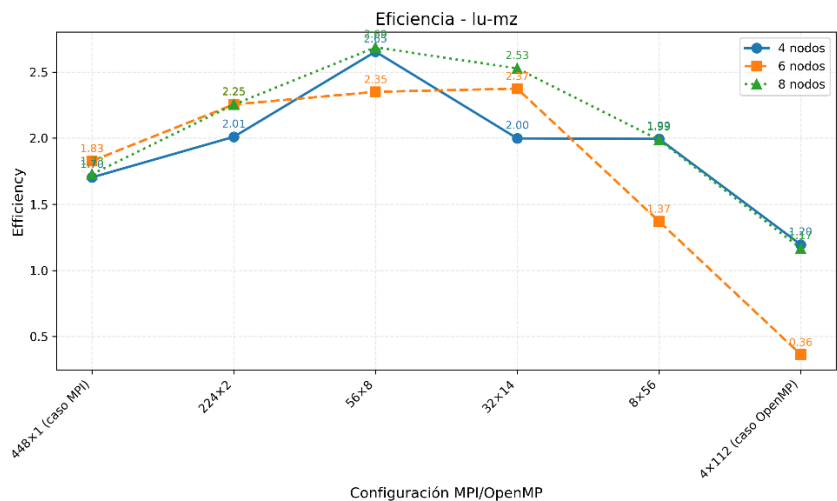


Ilustración 8 - Eficiencia de lu-mz

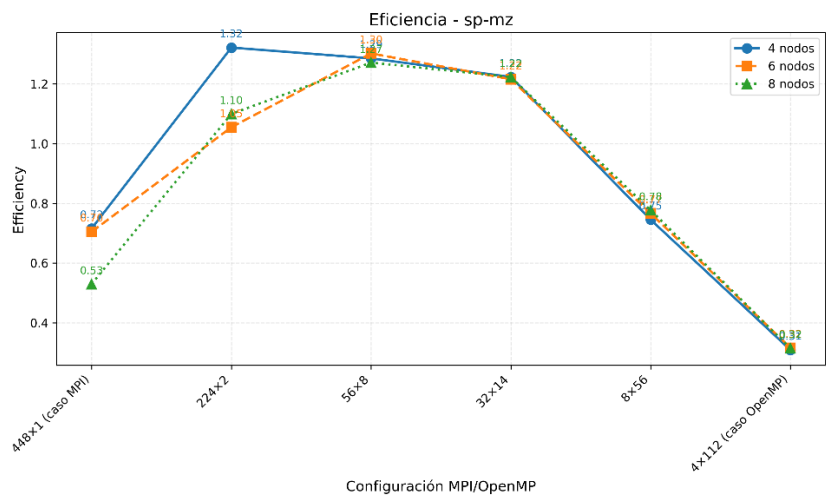


Ilustración 9 - Eficiencia de sp-mz

La eficiencia se ha calculado teniendo en cuenta que la ejecución secuencial es la ejecución llenando un nodo. Es decir, el parámetro es el número de nodos, que es entre lo que dividimos el speedup.

Se da un efecto distinto y no tan intuitivo como en las dos métricas anteriores. Si bien si que se puede observar que, en general, la eficiencia es mayor en los casos equilibrados, el número de nodos tiene un efecto distinto según el kernel. En bt-mz, es más eficiente usar menos nodos si usamos más MPI, similarmente a en sp-mz. En lu-mz parece que, en general, lo más eficiente es usar más nodos.

Lo que sí que podemos observar es que la eficiencia para los casos OpenMP es similar para cualquier número de nodos, lo que tiene sentido pues no debería alterar el funcionamiento de OpenMP usar más o menos nodos. Vemos como excepción el caso OpenMP con 6 nodos de lu-mz, que, tal y como hemos visto que es más lento, también es más ineficiente.

Conclusiones:

Sobre qué configuración es mejor, vemos que claramente la mejor opción es una bien balanceada, aunque este balance no es “mitad / mitad”, sino alrededor de unos 8 threads por task, para cualquiera de los tres kernels. Esta es la configuración que, en general, minimiza el tiempo de ejecución y maximiza el speedup y la eficiencia. Eso sí, hay alguna excepción. Por ejemplo en lu-mz, con 4 nodos, tarda ligeramente menos la configuración de 2 threads por task, que también es más eficiente.

Por lo tanto, si estamos obligados a escoger entre el caso MPI o el caso OpenMP, escoger el caso MPI será la opción más segura, y en general supondrá menor tiempo, mayor speedup y eficiencia, para cualquier kernel y número de nodos.

Entre los tres kernels hay claras diferencias de comportamiento. Bt-mz y sp-mz son similares, en cuanto a que usar una configuración u otra tiene un gran impacto en las métricas, así como usar más o menos nodos. Vemos que usar más nodos es, en general, más rápido y eficiente. Por otro lado, lu-mz no sufre tanto el efecto de usar una configuración u otra.

Para medir como de escalable es cada kernel podemos tratar de ver cómo es la evolución del tiempo, speedup y eficiencia conforme aumentamos los nodos. Usando este criterio, se puede observar que los tres kernels son más o menos igual de escalables, ya que las proporciones entre el ancho de las líneas es similar. Por ejemplo, si nos fijamos en el tiempo (figuras 1 – 3) vemos que la distancia entre las líneas de 6 y 8 nodos es aproximadamente la mitad que la distancia entre las líneas de 4 y 6 nodos.

Si marcamos un mínimo de un 0,7 para la eficiencia, habrá que descartar todos los casos OpenMP, y los casos MPI solo son factibles en lu-mz.

3. Análisis energético mediante EAR.

Resumen:

En este apartado se ha calculado el coste energético de los experimentos realizados, usando la potencia media, la energía en KWh (Kilo Watts hora) y el precio, suponiendo un coste de 0,2€ / KWh. Se incluyen gráficas de la potencia y la energía por configuración, número de nodos y kernel, y tablas con el consumo energético de cada script y del experimento en total, así como de su coste monetario.

Métricas:

Tras las ejecuciones, se obtiene la potencia en Watts para cada configuración de cada script ejecutado. Junto con el tiempo de cada ejecución, se pueden calcular también la energía y el coste monetario. Cabe recordar que estos valores están separados para el .csv de cada nodo.

Para la potencia se han utilizado indiscretamente todos los nodos de ambas repeticiones, y se ha normalizado respecto al máximo de 700 W que puede consumir cada CPU de las dos de un nodo.

En cuanto a energía, por un lado se ha calculado la media energética que consume cada configuración de cada script, como la media entre las dos repeticiones de la suma de todos sus nodos. Esto se visualiza en gráficas similares a las del resto de métricas, con una línea por número de nodos, un punto por configuración, y una gráfica por kernel.

Por otra banda, también se ha calculado el promedio de la energía que ha consumido cada script, como la suma de las seis medias que se muestran en el gráfico, es decir, la suma de los costes medios de cada configuración. Luego, se ha sumado el coste de los tres scripts de cada kernel, y por último se ha sumado el coste de los tres kernels y se ha multiplicado por las dos repeticiones. Todo esto se ha hecho también con el coste monetario, simplemente multiplicando por el supuesto coste de 0,2 € / KWh, y se ha colocado en las tablas 2 y 3.

No se ha calculado la energía, potencia ni coste monetario de las ejecuciones que llenan un solo nodo, ya que al ser tres scripts con una única configuración cada uno, es negligible al lado de nueve scripts con seis configuraciones cada uno (a pesar de haber hecho una repetición más).

Análisis de la potencia:

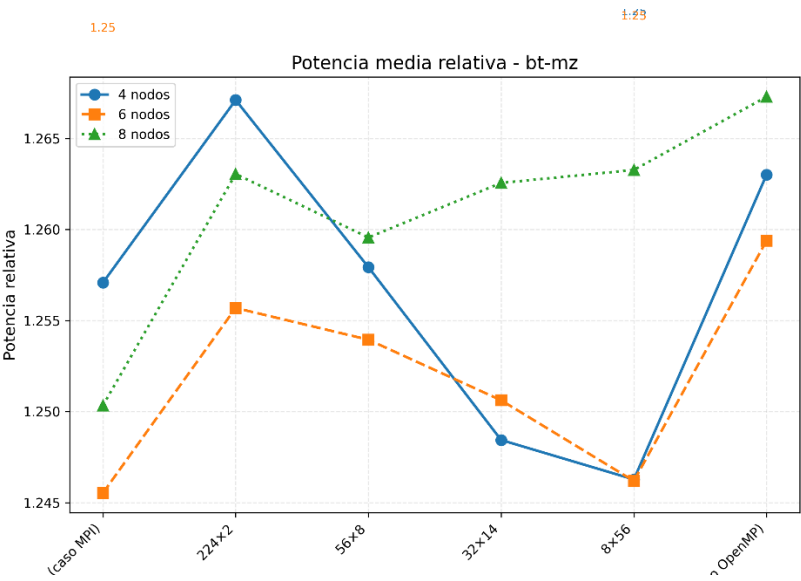


Ilustración 10 - Potencia media de bt-mz

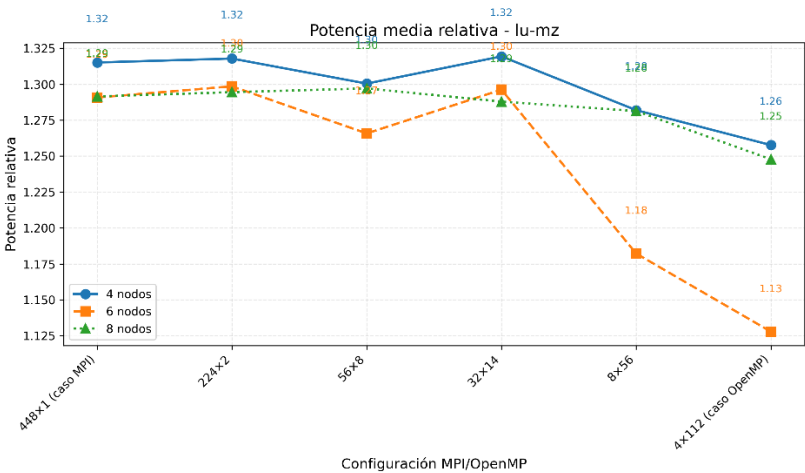


Ilustración 11 - Potencia media de lu-mz

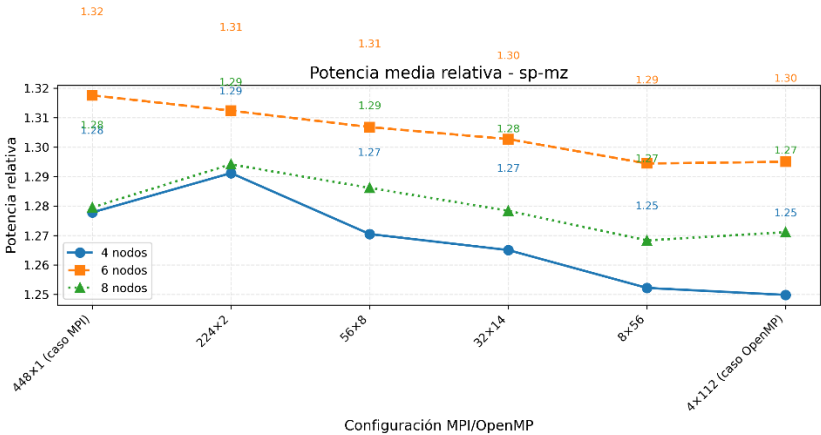


Ilustración 12 - Potencia media de sp-mz

A partir de la potencia es difícil sacar ninguna conclusión clara. Por un lado, parece que en lu-mz y sp-mz, la potencia tiende a bajar conforme usamos más threads por task, sin embargo, en bt-mz este efecto no se da hasta el caso OpenMP, sino que la potencia incrementa a partir de un punto. Si bien cada caso tiene una potencia diferente, no parece haber una relación lineal entre el número de nodos y la potencia. Por ejemplo, en sp-mz, vemos que con seis nodos hay más potencia media que con 4 u 8. Posiblemente este comportamiento se debe a factores que no se han tenido en cuenta en el experimento y se apaciguarían haciendo más repeticiones para sacar la media.

Análisis de la energía:

Los siguientes gráficos muestran la suma de la energía consumida por todos los nodos que han ejecutado cada configuración. Cada configuración es un punto.

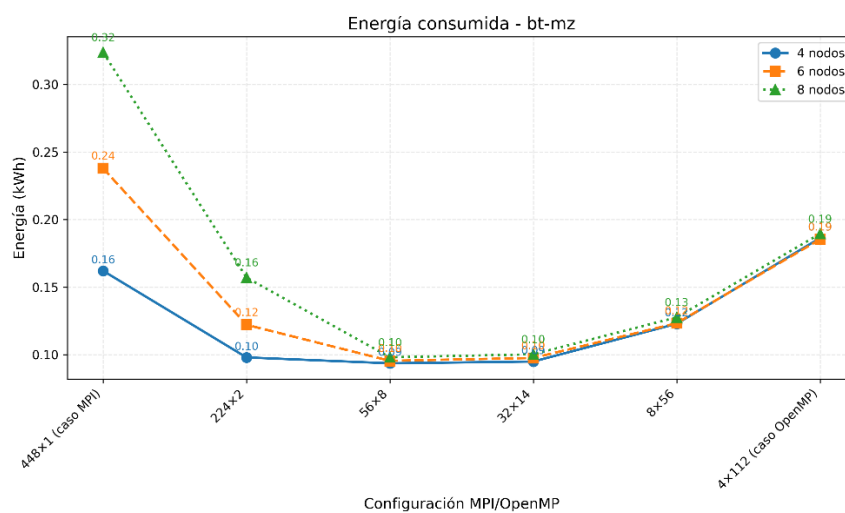


Ilustración 13 - Energía consumida por bt-mz

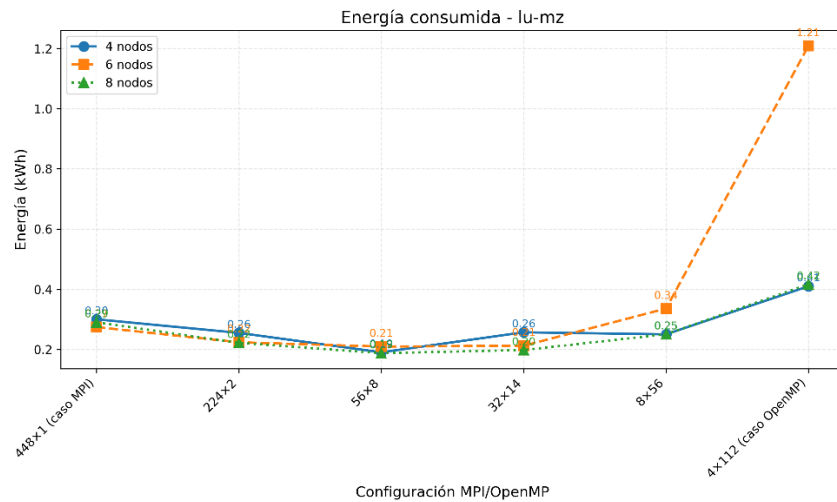


Ilustración 14 - Energía consumida por lu-mz

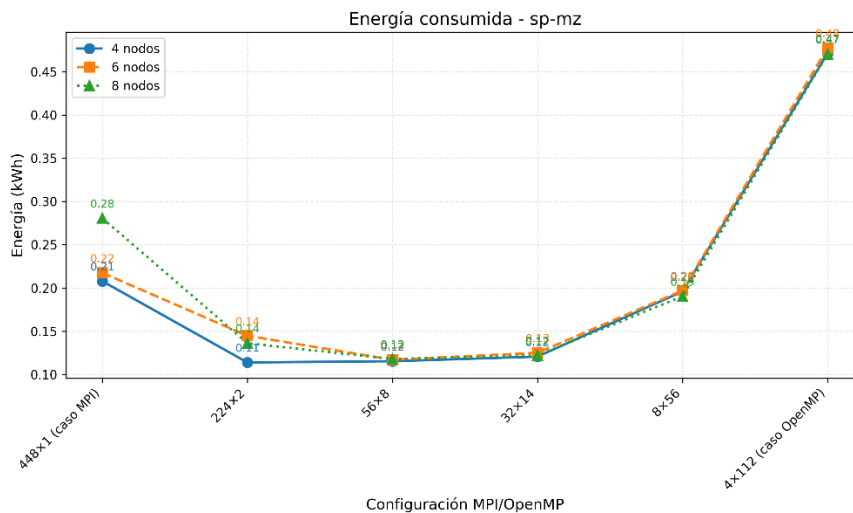


Ilustración 15 - Energía consumida por sp-mz

El comportamiento esperado es que, a mayor número de nodos, mayor consumo energético, pues los nodos van a potencias medias similares. Sin embargo, es clave observar las ilustraciones 1, 2 y 3, y notar que a mayor número de nodos, menor tiempo. La diferencia en tiempo se compensa con diferencia en potencia total, es decir, más nodos realizan el trabajo antes, pero el trabajo es el mismo. Por eso por lo general la energía consumida es la misma.

Vemos alguna excepción. En lu-mz, con 6 nodos, el caso OpenMP dispara el consumo, así como el tiempo de ejecución de lu-mz también se dispara con OpenMP (ilustración 2). Además, para bt-mz en el caso MPI, el tiempo era el mismo para 4, 6 u 8 nodos, lo que explica que veamos más consumo cuantos más nodos usados.

Tabla de consumo energético medio:

N v kernel >	bt-mz	lu-mz	sp-mz
4	0,76	1,67	1,22
6	0,86	2,47	1,28
8	1	1,57	1,32
Suma	2,62	5,7	3,82

Tabla 2: media del consumo energético (KWh) por kernel y número de nodos

Media de la energía consumida por cada repetición: 12,14KWh

Energía total consumida por las dos repeticiones: 24,27KWh

Tabla de coste monetario medio:

N v kernel >	bt-mz	lu-mz	sp-mz
4	0,15	0,33	0,24
6	0,17	0,49	0,25
8	0,2	0,31	0,26
Suma	0,52	1,14	0,76

Tabla 3: media del coste monetario (€) por kernel y número de nodos

Media del coste de una repetición: 2,43€

Coste total de las dos repeticiones: 4,85€

En promedio, cada script consume de energía los valores que muestra la tabla 2. Es decir, los scripts rondan entre 0,76 y 2,47 KWh usados, que se corresponden con costes entre 0,15 y 0,49 €.

En total, cada repetición consume unos 12,14 KWh, que cuesta unos 2,43€. Por lo tanto, habiendo ejecutado dos repeticiones, el experimento cuesta unos 24,27 KWh, o 4,85€. Cabe mencionar de nuevo que falta el coste de las ejecuciones para un solo nodo, pero podemos estar seguros de que no se llegaría a los 5€ para el experimento total, dado que son solo tres configuraciones ejecutadas tres veces.

4. Escalabilidad y métricas de rendimiento

Resumen:

En este apartado se ha evaluado el CPI y el ancho de banda de memoria de cada configuración para cada tamaño de nodos y kernel, con el objetivo principal de analizar si aumentar o reducir el número de procesos afecta a estas métricas.

Métricas:

Se han usado, para cada configuración de cada script, todos los nodos (de ambas repeticiones) indiscretamente para calcular la media, tanto de CPI como de ancho de banda de memoria.

Análisis del CPI:

Según los archivos .csv, el CPI es 1 en todas las ejecuciones, y también se puede verificar que el número de ciclos siempre es igual al número de instrucciones, que son iguales al número de tareas de cada configuración.

Esto puede ser causa por algún error en ear, en la realización del experimento, o simplemente porque los kernels crean un entorno perfectamente ideal y sin esperas. En cualquier caso, esta métrica no tiene sentido estudiarla en este caso.

Análisis del ancho de banda de memoria:

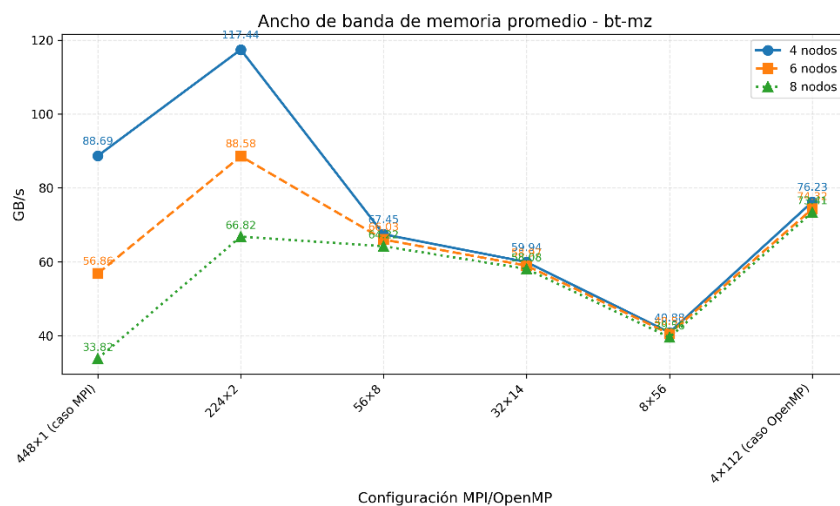


Ilustración 16 - Ancho de banda de memoria de bt-mz

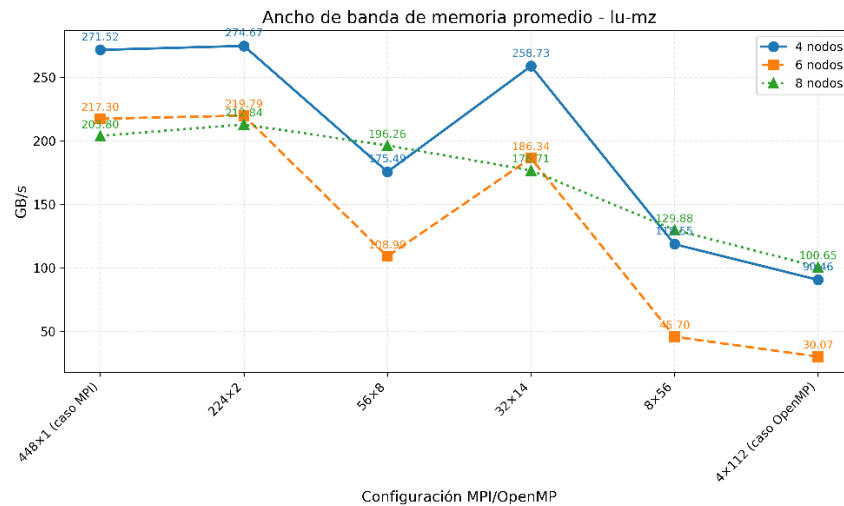


Ilustración 17 - Ancho de banda de memoria de lu-mz

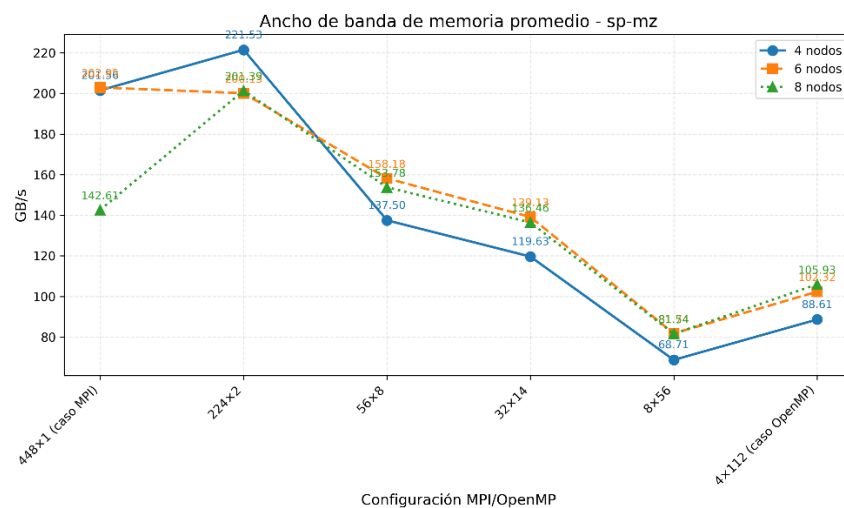


Ilustración 18 - Ancho de banda de memoria de sp-mz

Como es de esperar, cuanto más se usa OpenMP y menos MPI, tiende a haber menos ancho de banda de memoria, pues el paralelismo se da a nivel de tareas, más cercanas en memoria y que comparten mejor la cache. También vemos que bt-mz es el kernel que menos ancho de banda usa.

Par cada kernel el comportamiento es distinto. En bt-mz, el ancho de banda tiende a ser el mismo indiferentemente del número de nodos conforme nos acercamos al caso OpenMP. Para el caso MPI, hay menos ancho de banda conforme usamos más nodos. En lu-mz y sp-mz no se da esta convergencia, sino que la diferencia es más bien constante, excepto 8 nodos en lu-mz, que tiene un comportamiento distinto cuatro o seis nodos, y el caso MPI de sp-mz, en que el valor con seis nodos es más alto de lo que se esperaría.

5. Algoritmos y métricas de GPUs

Resumen:

Se ha hecho el mismo análisis energético que en el apartado 2, pero esta vez para los experimentos con tensorflow de la sesión de laboratorio de la asignatura de GPUs. Se incluye también el análisis del power de la GPU, normalizado a 700 W.

Experimentos:

En la sesión de laboratorio de GPUs se ejecutó el script `run_tensor.sh`, que es un script que contiene diferentes modelos con diferentes precisiones. Se han recuperado los datos de output de esa sesión de laboratorio, que incluyen la potencia en Watts del nodo y el tiempo en s, y se han normalizado a 700 W.

Análisis:

Modelo	Potencia (W)	Potencia normalizada	Tiempo (elapsed) (s)	Energía (KWh)
ResNet50	670,38	0,96	247	0,0460
ResNet50_mixed	677,96	0,97	253	0,0476
ResNet50_disable-tf32	667,22	0,95	288	0,0534
VGG19	654,11	0,93	189	0,0343
VGG19_mixed	650,87	0,93	95	0,0172
VGG19_disable-tf32	665,9	0,95	281	0,052
DenseNet121	654,82	0,94	331	0,0602
DenseNet121_mixed	664,32	0,95	345	0,0637
DenseNet121_disable-tf32	671,67	0,96	363	0,0677

Tabla 4: potencias en W y media de cada modelo de tensorflow

La energía total consumida por estas ejecuciones es de 0,44212KWh, que se corresponden con un coste de 0,08842€.

6. Reparto del trabajo

La redacción y toma de decisiones se ha hecho en conjunto por los dos miembros. Se ha repartido tan solo el trabajo no esencial, como puede ser el enviar a ejecutar los scripts, apuntar los jobIDs, organizar los archivos y programar el archivo de Python.